

CerbaSeal

Core v0.1.0

System & Repository Breakdown

Deterministic execution governance infrastructure for consequential
AI-assisted workflows. Fail-closed by design.

419

Tests

12

Invariants

16

Audit Checks

8

Starter Kits

17

Reason Codes

Contents

CerbaSeal-Core v0.1.0 — System & Repository Breakdown

1.	Project Overview & Purpose	
2.	Repository Structure	
3.	Architecture	
4.	Core Services	
5.	Domain Model & Types	
6.	Configuration & Policy	
7.	The 12 Core Invariants	
8.	Reason Codes	
9.	Integration Starter Kits (8 kits)	
10.	Audit & Evidence System	
11.	CLI Scripts & pnpm Commands	
12.	Test Coverage	
13.	Deployed Artifacts	

What CerbaSeal is and why it exists

CerbaSeal is a structural enforcement layer for consequential AI-assisted workflows. It answers one question at every decision point: may this proposed action proceed? The answer is always one of three deterministic outcomes — ALLOW, HOLD (pending human review), or REJECT — and every decision is recorded in a tamper-evident, hash-linked audit log.

The core design philosophy is organised around invariants — non-negotiable rules that cannot be overridden by configuration, policy, or caller behaviour — rather than features. AI systems are explicitly restricted to proposal-only roles; they can never self-authorise.

Key Design Principles

- **Fail-closed** Any unrecognised or malformed input produces REJECT, never silent pass-through.
- **AI non-authoritative** AI actors may propose actions but can never carry final authority. This is an unconditional invariant (INV-05).
- **gate.evaluate() never throws** All internal errors are caught and returned as structured GateResult with REJECT or HOLD.
- **Deterministic & replayable** Decisions can be re-evaluated against the same request via ReplayService for independent verification.
- **No silent fallbacks** Every failure mode surfaces as an explicit reason code on the DecisionEnvelope.

Maturity & Scope

CerbaSeal-Core v0.1.0 is described as a review-ready enforcement spine — architecturally real and covered by 419 tests, but not yet production-hardened. It is intended for pilot integrations, compliance demonstrations, and partner evaluation.

Top-level directory layout

cerbaseal-review/	
% % % src/	Core enforcement library
% % % % config/	CerbaSeal config & policy loaders
% % % % domain/	Types, constants, formatters, builders
% % % % services/	7 service modules (gate, evidence, audit, ...)
% % % examples/	8 integration starter kits + shared utilities
% % % % INTEGRATION-GUIDE.md	Decision tree for choosing the right kit
% % % % .../	One directory per kit
% % % artifacts/	Deployed web applications
% % % % cerbaseal-demo/	Partner portal & live demo (React + Vite)
% % % tests/	419 tests across 15 test files
% % % scripts/	Repo-audit, proof export/verify, generators
% % % docs/	Pilot documents, pricing, commercial framework
% % % cerbaseal.policy.json	Default policy pack (parseable at runtime)
% % % package.json	pnpm workspace root
% % % tsconfig.json	TypeScript project configuration

Architectural Boundaries (enforced by audit:repo)

- tests/ may not import from src/ directly (only via public surface).
- examples/ may not import private internals; only the public service API.
- No circular imports within src/.

Seven layers from input to evidence

1 Input Layer

Schema validation, actor authority class resolution, loggingReady check.

2 Decision Layer

Maps proposals (AI / Deterministic) to policy behaviours; enforces AI non-authority rule.

3 Execution Gate

Single entry point. Evaluates all 12 invariants. Returns GateResult (ALLOW / HOLD / REJECT).

4 Audit Layer

Append-only, SHA-256 hash-linked log. Every event produces a chain-linked AuditLogEntry.

5

Evidence Layer

Assembles EvidenceBundle: request + DecisionEnvelope + ReleaseAuthorization + eventChain.

6

Export / Replay Layer

ReplayService re-runs original requests to verify recorded outcomes still hold.

7

Support / Diagnostic

DiagnosticReport, OperatorActionReport — human-readable analysis of gate decisions.

ExecutionGateService — Primary enforcement engine

`src/services/execution/`

Evaluates GovernedRequest objects against all 12 invariants in strict order. The single point of entry for all governed actions. Returns GateResult containing DecisionEnvelope plus either ReleaseAuthorization (ALLOW) or BlockedActionRecord (HOLD/REJECT). Never throws — internal errors surface as structured REJECT results.

EvidenceBundleService — Decision artifact assembly

`src/services/evidence/`

Assembles immutable EvidenceBundle snapshots from a GateResult and GovernedRequest. Records 3–4 audit events per evaluation (REQUEST_EVALUATED, RELEASE_AUTHORIZED or ACTION_BLOCKED, EVIDENCE_BUNDLE_CREATED). Rejects GateResult objects not produced by ExecutionGateService.evaluate() to prevent self-constructed forgeries.

AppendOnlyLogService / FileBackedAppendOnlyLogService — Hash-linked audit log

`src/services/audit/`

Maintains the SHA-256 hash chain of AuditLogEntry records. In-memory implementation for tests; file-backed implementation for deployments. Each entry stores: eventId, requestId, eventType, timestamp, payloadHash, previousHash, entryHash. Chain integrity is verifiable without the original payloads.

DiagnosticReportService — Decision analysis

`src/services/diagnostics/`

Generates DiagnosticReport artifacts containing status, severity (INFO / WARNING / CRITICAL), operator explanation, and per-invariant pass/fail detail. Used by the partner portal Operator View and by support-readiness validation.

SystemIntegrityService / OperatorActionService — System health & operator guidance

`src/services/support/`

SystemIntegrityService runs internal verification scenarios to confirm predictable gate behaviour. OperatorActionService translates gate results into actionable remediation steps for human operators.

ReplayService — Decision verification

`src/services/replay/`

Re-runs original GovernedRequest objects through the gate and compares outcomes to previously recorded DecisionEnvelopes. Enables independent auditors to verify that recorded decisions remain consistent with current enforcement logic.

ExportManifestService — Proof & export

`src/services/export/`

Generates ExportManifest artifacts and cryptographic proof snapshots of the current system state. Used by pnpm `export:proof` / `pnpm verify:proof` workflows.

Primary Types (src/domain/types/core.ts)

- **GovernedRequest** The gate's input. Contains: requestId, workflowClass, jurisdiction, actorId, actorAuthorityClass, proposal (AI/Deterministic), sensitive, prohibitedUse, policyPackRef, provenanceRef, approvalRequired, approvalArtifact, loggingReady, controlStatus, trustState.
- **GateResult** Gate output. Contains: decisionEnvelope, releaseAuthorization (or null), blockedActionRecord (or null), diagnosticReport, triggeredInvariants.
- **DecisionEnvelope** Immutable outcome record. Contains: envelopedId, evidenceBundleId, requestId, finalState (ALLOW/HOLD/REJECT), workflowClass, reasonCodes[], immutableSignature, decidedAt.
- **ReleaseAuthorization** The authoritative permission token. Present only on ALLOW. Contains: releaseAuthorizationId, requestId, actionClass, authorizedBy, releaseSignature, expiresAt, issuedAt.
- **BlockedActionRecord** Present on HOLD and REJECT. Contains: requestId, finalState, reasonCodes[], blockedAt.
- **ApprovalArtifact** Evidence of human review. Contains: approvalId, approverId, forRequestId, approverAuthorityClass, privilegedAuthSatisfied, immutableSignature, approvedAt.
- **ApprovalProposal** AI or deterministic proposal embedded in GovernedRequest. Contains: proposalSourceKind, authorityBearing, requestedActionClass, confidence, reasonCodes, proposalCreatedAt.

Audit Types (src/domain/types/audit.ts)

- **AuditLogEntry** Hash-chain node: eventId, requestId, eventType, timestamp, payloadHash, previousHash, entryHash.
- **AuditEventType** Union: "REQUEST_EVALUATED" | "RELEASE_AUTHORIZED" | "ACTION_BLOCKED" | "EVIDENCE_BUNDLE_CREATED" | "EXPORT_MANIFEST_CREATED".
- **EvidenceBundle** Self-contained decision artifact: evidenceBundleId, request, decisionEnvelope, releaseAuthorization, blockedActionRecord, eventChain, createdAt.
- **ExportManifest** Proof export record: manifestId, evidenceBundleId, requestId, envelopedId, exportedAt, schemaVersion.

Authority Classes

The gate recognises six actor authority classes, defined in cerbaseal-config.ts:

- **system** Automated governance processes. May perform most actions without human approval.
- **ai** AI models and agents. Proposal-only. Cannot self-authorise under any circumstances (INV-05).
- **analyst** Human analysts. May approve certain workflows when explicit ApprovalArtifact is provided.
- **reviewer** Compliance reviewers. Elevated approval authority.
- **manager** Management-level approvers.
- **compliance_officer** Highest authority class for approval chains.

CerbaSealConfig (cerbaseal-config.ts)

System-wide settings loaded by ExecutionGateService. Defines the set of valid authority classes and hardcoded workflow constraints.

- **WORKFLOWS_REQUIRING_APPROVAL** Hardcoded set. Currently includes "fraud_triage". Always requires human ApprovalArtifact regardless of actor class.
- **authorityClasses** Registered set of valid actorAuthorityClass values.

CerbaSealPolicy (cerbaseal-policy.ts + cerbaseal.policy.json)

Optional client-specific policy layer applied on top of the invariant core. Parsed from cerbaseal.policy.json at runtime (check 16 in audit:repo verifies it parses without error).

- **actorMappings** Map client role labels to core authority classes, e.g. "VP Risk" != "manager".
- **approvalChains** Define which authority classes may approve specific workflow/action combinations.
- **actionPolicies** Per workflow+action rules: "requires_approval", "auto_allow", or "blocked".

loadCerbaSealConfig()

Factory function that returns the active CerbaSealConfig. Used by ExecutionGateService, all validation scripts, and the demo portal.

Non-negotiable enforcement rules — cannot be overridden by config or caller

Invariants are evaluated in strict order by ExecutionGateService. The first violated invariant determines the outcome. 100% of invariants are covered by tests (12/12). Invariant codes are registered in src/domain/constants/invariants.ts.

Code	Invariant	Condition that triggers it	Outcome
INV-01	No Policy Pack !' No Execution	policyPackRef missing or null	REJECT
INV-02	No Provenance !' No Action	provenanceRef missing or null	REJECT
INV-03	No Required Approval !' No Release	approvalRequired=true but no valid artifact	HOLD
INV-04	No Logging !' No Execution	loggingReady=false	REJECT
INV-05	AI Non-Authoritative	actorAuthorityClass='ai'	REJECT
INV-06	No Bypass of Execution Gate	Structural — enforced by module boundaries	N/A
INV-07	Immutable Decision Envelope	Structural — DecisionEnvelope sealed at issue	N/A
INV-08	Stale Controls Block Sensitive Release	controlStatus.stale=true on sensitive request	REJECT
INV-09	Trust State Required	trustState.trusted=false	REJECT
INV-10	Prohibited Use Must Block	prohibitedUse=true	REJECT
INV-11	Request Schema & Action Class Valid	Unknown proposedActionClass or malformed req	REJECT
INV-12	Proposal & Request Action Must Match	proposal.requestedActionClass != proposedActionClass	REJECT

Hardcoded Workflow Constraints

- **fraud_triage workflow** ALWAYS requires human approval regardless of actor class or policy. Encoded in WORKFLOWS_REQUIRING_APPROVAL constant.
- **Approval authority check** ApprovalArtifact.privilegedAuthSatisfied must be true. Approver authority class must meet the chain requirement.

Every `GateResult` carries one or more reason codes on the `DecisionEnvelope`. Codes are stable string constants — safe to match in downstream code.

DECISION_ALLOWED	ALLOW	Request passed all invariant checks.
DECISION_HELD	HOLD	Approval required; withheld pending human review.
DECISION_REJECTED	REJECT	One or more invariants violated; action blocked.
NO_POLICY_PACK	REJECT	policyPackRef missing (INV-01).
NO_PROVENANCE	REJECT	provenanceRef missing (INV-02).
REQUIRED_APPROVAL_MISSING	HOLD	approvalRequired=true but no ApprovalArtifact (INV-03).
INVALID_APPROVAL_AUTHORITY	HOLD	Approver authority class insufficient.
PRIVILEGED_AUTH_NOT_SATISFIED	HOLD	ApprovalArtifact.privilegedAuthSatisfied=false.
APPROVAL_SIGNATURE_MISSING	HOLD	ApprovalArtifact.immutableSignature absent.
LOGGING_NOT_READY	REJECT	loggingReady=false (INV-04).
AI_CANNOT_AUTHORIZE	REJECT	Actor is 'ai' authority class (INV-05).
PROHIBITED_USE	REJECT	prohibitedUse=true (INV-10).
CONTROL_STALE_OR_INVALID	REJECT	Stale controls on sensitive request (INV-08).
TRUST_STATE_INVALID	REJECT	trustState.trusted=false (INV-09).
UNKNOWN_ACTION_CLASS	REJECT	proposedActionClass not registered (INV-11).
MALFORMED_REQUEST	REJECT	Request structure invalid (INV-11).
INVALID_PROPOSAL	REJECT	Proposal/request action mismatch (INV-12).

Each kit is a self-contained directory with `index.ts` (the integration pattern), `validate-*.ts` (runnable smoke test), and `README.md`. All validate scripts produce 100% PASS on a clean checkout. See `INTEGRATION-GUIDE.md` for the decision tree routing clients to the right kit.

Fraud Workflow Starter

`examples/fraud-workflow-starter/`

End-to-end reference: `GovernedRequest` → `gate.evaluate()` → `EvidenceBundleService` → `DiagnosticReport`.

The canonical starting point for new integrations.

Financial Approval

`examples/financial-approval-starter/`

Multi-step approval flow. AI proposes → analyst reviews → manager approves. Demonstrates

`ApprovalArtifact` construction and `HOLD` → `ALLOW` lifecycle.

Agent Gate

`examples/agent-gate / agent-integration-starter/`

MCP-style pattern. CerbaSeal gates tool-execution calls from an AI agent before any action is dispatched. Shows how AI agents stay proposal-only.

Consumer Example

`examples/consumer-example/`

Minimal direct integration. Import `gate`, build `GovernedRequest`, call `evaluate()`, handle all three outcomes. Lowest-friction entry point.

Auditor View

`examples/auditor-view/`

Renders hash-linked audit certificates for external reviewers. Demonstrates how `EvidenceBundle` is formatted as a human-readable compliance certificate.

Webhook Adapter

`examples/webhook-adapter/`

Node.js HTTP server. `POST /event` → `gate.evaluate()` → async `POST` to `CALLBACK_URL`. No framework dependency. Handles all three outcomes; returns 202 immediately.

Express Middleware

`examples/express-middleware/`

`cerbaSealGate()` returns an Express-compatible middleware. `ALLOW` → `next()` + `res.locals`. `REJECT` → 403 + reason codes. `HOLD` → 202 + `envelopeId`. Custom `onHold()` supported.

Async Queue

`examples/async-queue/`

`JobQueue` class. `enqueue()` evaluates immediately. `approve()` attaches `ApprovalArtifact` and re-evaluates. Pure in-memory — swap internals for `Bull/BullMQ/SQS` without touching gate logic.

Audit Export

`examples/audit-export/`

Reads `FileBackedAppendOnlyLogService` JSONL. Verifies chain integrity. Infers outcomes from event types. Optional `EvidenceBundle` enrichment adds workflow/actor grouping.

REST API Starter

`examples/rest-api-starter/`

Thin HTTP API wrapper showing how to expose gate evaluation over a REST endpoint, with request parsing, response shaping, and error handling.

Support Readiness

`examples/support-readiness/`

Demonstrates `SystemIntegrityService` and `OperatorActionService`. Generates `DiagnosticReport` for a `HOLD` decision and produces actionable operator guidance.

Hash Chain Algorithm

Every audit event is recorded as an `AuditLogEntry`. Each entry's `entryHash` is computed from: `eventId + requestId + eventType + timestamp + payloadHash + previousHash`. The payload itself is hashed (SHA-256) but not stored inline, so the chain can be verified from the JSONL file alone without the original request data.

Events per Gate Evaluation

- **REQUEST_EVALUATED** Always written. Payload: `requestId, finalState, enveloped`.
- **RELEASE_AUTHORIZED** ALLOW outcomes only. Payload: `releaseAuthorizationId, actionClass`.
- **ACTION_BLOCKED** HOLD / REJECT outcomes only. Payload: `finalState, reasonCodes[]`.
- **EVIDENCE_BUNDLE_CREATED** Always written. Payload: `evidenceBundleId`.

FileBackedAppendOnlyLogService

The file-backed implementation appends one JSON line per event to a JSONL file. Each line is a full `AuditLogEntry`. The in-memory `AppendOnlyLogService` uses the same `buildEntry()` and `verifyChain()` logic, so chains produced by one are verifiable by the other.

EvidenceBundle

Self-contained decision artifact produced by `EvidenceBundleService.createBundle()`. Contains the full `GovernedRequest`, `DecisionEnvelope`, `ReleaseAuthorization` (or null), `BlockedActionRecord` (or null), and `eventChain` (`AuditLogEntry[]`). Suitable for export to compliance systems, independent auditors, or long-term archival. `EvidenceBundleService` rejects `GateResult` objects not issued by `ExecutionGateService.evaluate()` to prevent self-constructed forgeries.

Proof Snapshots

`pnpm export:proof` generates a cryptographic proof snapshot of the current system state. `pnpm verify:proof` validates a snapshot against the live codebase. Used for 'Proven at Runtime' claims in compliance documentation.

Development, validation, and maintenance commands

`pnpm test`

Run all 419 tests (Vitest).

`pnpm audit:repo`

Run all 16 repo-audit health checks: tests, tsc, docs, routes, invariants, boundaries, pilot docs, policy file.

`pnpm check:imports`

Enforce architectural import boundaries (no tests!'src, no examples!'private).

`pnpm check:invariants`

Verify every registered invariant has at least one covering test.

`pnpm export:proof`

Generate a cryptographic proof snapshot of the current system state.

`pnpm verify:proof`

Validate a previously exported proof against the live codebase.

`pnpm demo:web`

Start the partner portal (React + Vite) demo application.

`pnpm demo:web:validate`

Run assertions against all partner portal routes.

`pnpm demo:support:validate`

Validate 13 support-readiness scenarios.

`pnpm review:validate`

Run 110 assertions covering the review portal and all gate outcomes.

`pnpm tsx examples/<kit>/validate-*.ts`

Run a specific starter kit's validation script.

Repo-Audit Checks (16 / 16 PASS)

- ' 1. Full test suite passes
- ' 2. TypeScript compiles without errors (tsc --noEmit)
- ' 3. README anchor strings present
- ' 4. All portal routes respond 200
- ' 5. No src/ file unreferenced in tests or examples
- ' 6. Invariant registry exists and is non-empty
- ' 7. Known-limitations section in README
- ' 8. Test count in README matches actual
- ' 9. demo:web:validate passes
- ' 10. demo:support:validate passes
- ' 11. review:validate passes
- ' 12. No architectural import boundary violations
- ' 13. All invariants linked to covering tests
- ' 14. No stale test-count references in documentation
- ' 15. Pilot documents exist with required sections
- ' 16. cerbaseal.policy.json parses without error

CerbaSeal's test suite is organised around invariants and adversarial scenarios rather than code paths. The goal is to verify that every forbidden state is correctly blocked, every allowed state is correctly released, and that no combination of inputs can bypass the gate logic.

419

Total tests (Vitest).

12/12

Invariants with at least one covering test — 100% coverage.

122+

Tests specifically targeting adversarial bypass, fail-closed behaviour, and non-forgery.

110

Assertions validated by review:validate across all gate outcomes and portal routes.

13

Assertions validated by demo:support:validate.

15

Test files total.

Test Categories

- **Invariant tests** One or more tests per invariant verifying the exact reject/hold behaviour.
- **Happy-path tests** Full ALLOW flows for system, analyst, reviewer, and manager authority classes.
- **Adversarial tests** Self-constructed GateResult forgery attempts, modified decision envelopes, invalid approval artifacts.
- **Chain-integrity tests** Tampered JSONL files, missing links, out-of-order entries.
- **Replay tests** Verifying that ReplayService detects outcome drift between recorded and re-evaluated decisions.
- **Portal validation** All 9 partner portal routes confirm 200 OK; all demo scenarios assert exact gate outcomes.

CerbaSeal Demo ([artifacts/cerbaseal-demo/](#))

The partner portal and live demo application. Built with React + Vite. Serves as the primary commercial-facing surface for CerbaSeal-Core v0.1.0.

- **Partner Portal (/partner)** Resource library with 21 items: pilot docs, starter kits, integration guide. Lists all 8 integration kits and links to the decision tree.
- **Reviewer View** Shows EvidenceBundle certificates rendered for external auditors. Demonstrates hash-chain output.
- **Operator View** Displays DiagnosticReport for HOLD and REJECT decisions with actionable remediation steps.
- **Demo Scenarios** Interactive walkthrough of all three gate outcomes across multiple workflow classes.
- **9 routes, all 200 OK** Verified by audit:repo check #4 on every run.

Pilot Documents ([docs/](#))

Three pilot documents are verified by audit:repo check #15:

- Pilot onboarding guide
- Commercial framework / pricing reference
- Integration readiness checklist

CerbaSeal-Core v0.1.0

System & Repository Breakdown

Deterministic execution governance for consequential AI-assisted workflows.

419 tests · 12 invariants · 16 audit checks · 8 integration starter kits

All tests PASS · audit:repo 16/16 PASS · tsc --noEmit clean

